

Fractional-Return AMMs with Newton-Based Arbitrage Rebalancing

Keyaan Gala, Kevin Pasato

EE/CpE 608 Applied Modeling and Optimization

GitHub: <https://github.com/Kpasato/AMM-Price-Alignment-EE608>

Abstract

Weighted automated market makers (AMMs) assign fixed pool weights to each asset, and those weights directly control the fractional return exposure the pool provides. In this project, we study whether AAPL/MSFT/cash pool weights can be chosen to match a target return profile, and whether Newton’s method can compute the trade needed to realign the AMM price with an external market price. We model this as a constrained least-squares problem in log-return space, fit weights using AAPL and MSFT daily returns from 2020 to 2025, and validate the results against three target profiles. We also implement a simplified one-dimensional Newton solver for price alignment. The results show that geometric targets are recovered exactly, arithmetic targets are approximated closely, and Newton’s method converges in a small number of iterations.

1 Introduction

Automated market makers are usually discussed in the context of decentralized finance, where liquidity providers deposit assets into a pool and traders interact with the pool via a pricing formula. The standard constant-product AMM uses equal 50/50 weights, but weighted variants allow the pool to hold assets in different proportions.

What is less commonly discussed is that these weights also determine how much of an asset’s return the pool actually captures. If a pool holds 50% AAPL and 50% cash, its gross return on any given day is approximately $R_{\text{AAPL}}^{0.5}$, the square root of AAPL’s gross return. This is a strictly fractional exposure: the pool gains less on up days, but also loses less on down days.

This observation suggests a natural optimization question: given a target return profile, which weights best replicate it? And once the weights are fixed, if the external market price of AAPL shifts, how large a trade is needed to bring the AMM price back into alignment?

These are the two questions this project addresses. The first is a constrained least-squares problem solved numerically. The second is a one-dimensional root-finding problem solved with Newton’s method.

2 Problem Description

We want to choose portfolio weights w_A , w_M , and w_C (for AAPL, MSFT, and cash respectively) such that the daily log-return of the weighted AMM tracks a target log-return as closely as possible. The weights must all be non-negative and sum to one, so they represent a valid long-only allocation.

Once the weights are chosen, we study a second problem: after external market prices move, the AMM's internal price (derived from its reserves and weights) may differ from the current market price. We use Newton's method to find the trade size that eliminates this gap.

Both problems are studied in an educational setting using historical stock data rather than live AMM pool data. The goal is to understand how the AMM structure connects weights, returns, and prices, and to apply optimization tools to both problems.

3 Dataset

We use daily adjusted close prices for AAPL and MSFT downloaded via `yfinance` with `auto_adjust=True`. The date range is January 1, 2020 through December 31, 2025.

Gross daily returns are computed as:

$$R_t = \frac{P_t}{P_{t-1}}$$

Log returns are then computed as $\log R_t$. After removing the first row (which has no prior day), this gives approximately 1,500 daily observations per ticker.

4 Fractional AMM Return Model

A weighted AMM with pool weights w_A , w_M , and w_C (AAPL, MSFT, cash) produces a gross return at time t of:

$$R_{\text{AMM},t} = R_{\text{AAPL},t}^{w_A} \cdot R_{\text{MSFT},t}^{w_M} \cdot R_{\text{cash},t}^{w_C}$$

Since cash earns a gross return of 1 (ignoring interest), this simplifies to:

$$R_{\text{AMM},t} = R_{\text{AAPL},t}^{w_A} \cdot R_{\text{MSFT},t}^{w_M}$$

For a pool with only AAPL and cash:

$$R_{\text{AMM},t} = R_{\text{AAPL},t}^{w_A}$$

At equal weights $w_A = 0.5$:

$$R_{\text{AMM},t} = \sqrt{R_{\text{AAPL},t}}$$

Taking logs, the AMM log-return is a linear combination of the asset log-returns:

$$\log R_{\text{AMM},t} = w_A \log R_{\text{AAPL},t} + w_M \log R_{\text{MSFT},t}$$

This is why the weights act directly as return exposures in log space. A weight of 0.5 on AAPL means the pool captures half of AAPL's log-return each day.

5 Optimization Model

Decision Variables

The decision variables are the pool weights:

$$w_A \in [0, 1], \quad w_M \in [0, 1], \quad w_C = 1 - w_A - w_M$$

Objective Function

Given a target log-return series $\{\ell_t^*\}$, we minimize the sum of squared tracking errors over all T trading days:

$$\min_{w_A, w_M} \sum_{t=1}^T (\ell_t^* - w_A \log R_{\text{AAPL},t} - w_M \log R_{\text{MSFT},t})^2$$

Constraints

$$\begin{aligned} w_A &\geq 0 \\ w_M &\geq 0 \\ w_A + w_M &\leq 1 \\ w_C &= 1 - w_A - w_M \end{aligned}$$

Rationale

The objective measures how well the AMM log-return tracks the target in log space. This is a natural fit because the AMM return formula is multiplicative, and logs convert it to a linear combination of the weights. Minimizing squared log-return errors is equivalent to minimizing the squared deviation of the AMM's daily return exponent from the target.

The constraints enforce long-only positions across AAPL, MSFT, and cash. The cash term absorbs any unused weight, making the allocation explicit. The constraint $w_A + w_M \leq 1$ prevents leverage: the pool cannot have more than 100% invested in the two risky assets.

Convexity

The objective is a quadratic function of (w_A, w_M) , specifically a sum of squared linear functions. This is convex. The constraints are all linear, so the feasible set is a closed, bounded triangle (a simplex) in two dimensions. Because the objective is convex and the feasible set is convex and compact, the optimization problem has a global minimum and is clear to solve. No local minima exist.

Numerical Method

We solve this problem using `scipy.optimize.minimize` with the SLSQP (Sequential Least Squares Programming) method. SLSQP was chosen because it directly supports both bound constraints and general inequality constraints, handles the problem cleanly without reformulation, and converges quickly for small-dimensional quadratic problems like this one.

6 Target Return Profiles

We test the optimizer against three target profiles to study when the AMM can match a target exactly and when it can only approximate it.

Square-Root AAPL Target

$$\ell_t^* = 0.5 \log R_{\text{AAPL},t}$$

This target is exactly representable by the AMM model with $w_A = 0.5$, $w_M = 0$, $w_C = 0.5$. We expect the optimizer to recover these weights with zero tracking error.

Mixed 40/40 Target

$$\ell_t^* = 0.4 \log R_{\text{AAPL},t} + 0.4 \log R_{\text{MSFT},t}$$

This target is also exactly representable with $w_A = 0.4$, $w_M = 0.4$, $w_C = 0.2$. Again, we expect exact recovery.

Simple 50/50 Arithmetic Blend

$$R_{\text{target},t} = 1 + 0.5(R_{\text{AAPL},t} - 1) + 0.5(R_{\text{MSFT},t} - 1)$$

$$\ell_t^* = \log R_{\text{target},t}$$

This target blends the two stocks equally in dollar terms rather than log terms. An arithmetic average of gross returns is not, in general, equal to a geometric mean with any fixed weights. The optimizer will find the weights that minimize tracking error, but the fit will not be perfect. This case is interesting because it shows the limits of what the AMM model can represent.

7 Newton-Based Price Alignment

AMM Price Formula

In a two-asset zero-fee weighted AMM with reserves x_i and x_j and weights w_i and w_j , the marginal price of token i in terms of token j is:

$$p_{i,j} = \frac{x_j w_i}{x_i w_j}$$

Trade Convention

When a trader deposits δ_i units of token i , the AMM pays out δ_j units of token j , computed from the swap invariant as:

$$\delta_j = x_j \left[1 - \left(\frac{x_i}{x_i + \delta_i} \right)^{w_i/w_j} \right]$$

After the trade, the updated reserves are:

$$x'_i = x_i + \delta_i, \quad x'_j = x_j - \delta_j$$

Post-Trade Price

The AMM price after the trade is:

$$p_{\text{new}}(\delta_i) = \frac{x'_j w_i}{x'_i w_j}$$

Root-Finding Problem

To align the AMM price with an external market price P^* , we solve:

$$f(\delta_i) = p_{\text{new}}(\delta_i) - P^* = 0$$

Newton's Method

Starting from an initial guess $\delta_i^{(0)} = 0$, Newton's update at each step is:

$$\delta_i^{(k+1)} = \delta_i^{(k)} - \frac{f(\delta_i^{(k)})}{f'(\delta_i^{(k)})}$$

The derivative f' is approximated numerically using the central-difference formula:

$$f'(x) \approx \frac{f(x + \varepsilon) - f(x - \varepsilon)}{2\varepsilon}, \quad \varepsilon = 10^{-6}$$

We include safety checks to ensure that $x_i + \delta_i > 0$ and that $\delta_j < x_j$ at every iteration. The solver stops when $|f(\delta_i)| < 10^{-10}$ or the derivative is too small to proceed safely.

This is a simplified educational implementation. It uses a zero-fee model and a single two-asset pool.

8 Implementation

The project is organized into the following Python files:

- `data/fetch_data.py` — downloads prices, computes gross returns, saves CSVs
- `model/amm_returns.py` — AMM return formula, cumulative growth, scenario builder
- `model/weights_optimizer.py` — constrained least-squares weight fitting
- `model/newton_solver.py` — Newton’s method for AMM price alignment
- `analysis/plots.py` — matplotlib plotting and CSV saving helpers
- `analysis/run_analysis.py` — runs the full pipeline end to end

All outputs are written to an `outputs/` directory. The pipeline runs with a single command: `python3 analysis/run_analysis.py`.

9 Results

Weight Optimization

Table ?? shows the optimized weights and tracking metrics for each target profile.

Table 1: Optimization results for each target return profile.					
Target	w_A	w_M	w_C	Max Abs Error	Correlation
<code>sqrtaapl</code>	0.500	0.000	0.500	0.000000	1.000000
<code>mixed4040</code>	0.400	0.400	0.200	0.000000	1.000000
<code>simple5050</code>	0.500	0.500	0.000	0.001107	0.999993

The first two targets are recovered exactly. The third target, which uses an arithmetic average of gross returns, is approximated closely but not exactly. The optimizer converges to equal AAPL/MSFT weights, which makes intuitive sense as a best-fit to a 50/50 blend.

Newton Solver

For a pool with $x_i = x_j = 1000$ and equal weights $w_i = w_j = 0.5$, the initial AMM price is 1.0. When the external market price drops to 0.9, Newton’s method finds the trade that realigns the AMM:

- $\delta_i = 54.092553$ (AAPL deposited into the pool)
- $\delta_j = 51.316702$ (MSFT received from the pool)
- Final AMM price: 0.9 (exact match to external price)
- Converged in 4 iterations

Plots

Figure ?? shows the cumulative growth of \$1 for each fractional weight scenario.

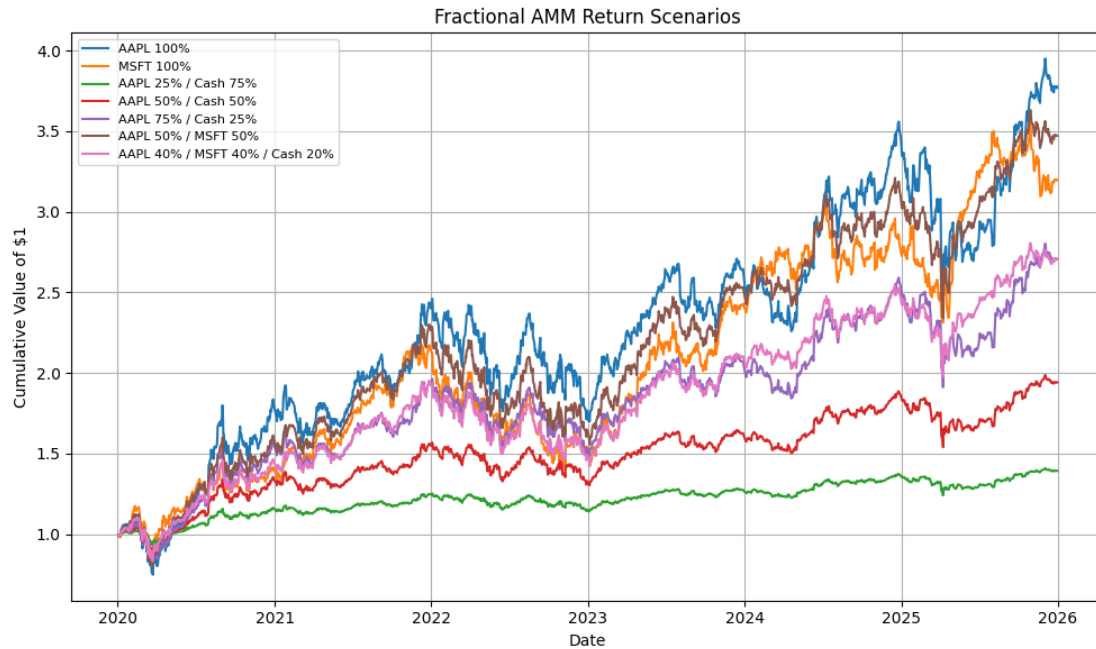


Figure 1: Cumulative growth of \$1 under several AAPL/MSFT/cash weight scenarios, 2020–2025.

Figures ??–?? show target vs. optimized AMM cumulative returns for each optimization target.

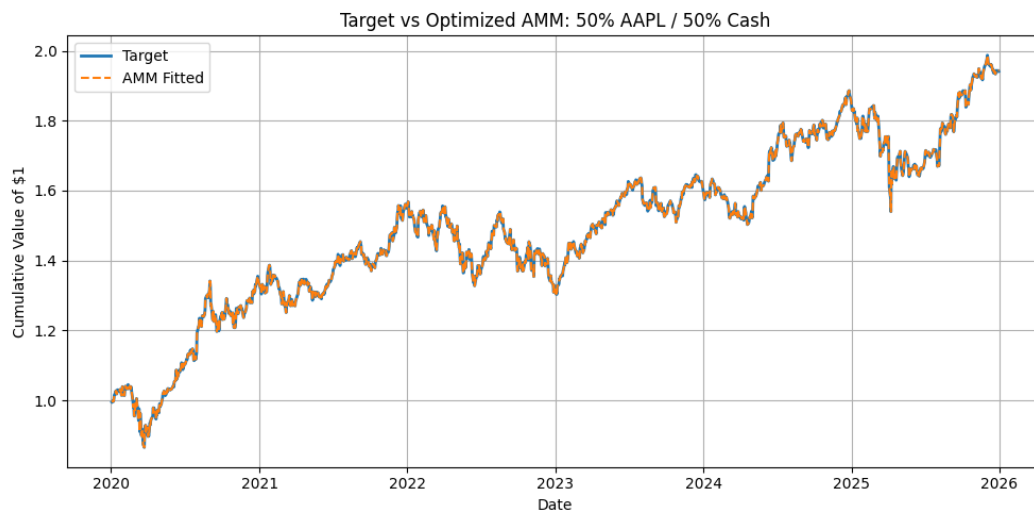


Figure 2: Target vs. optimized AMM: 50% AAPL / 50% cash.

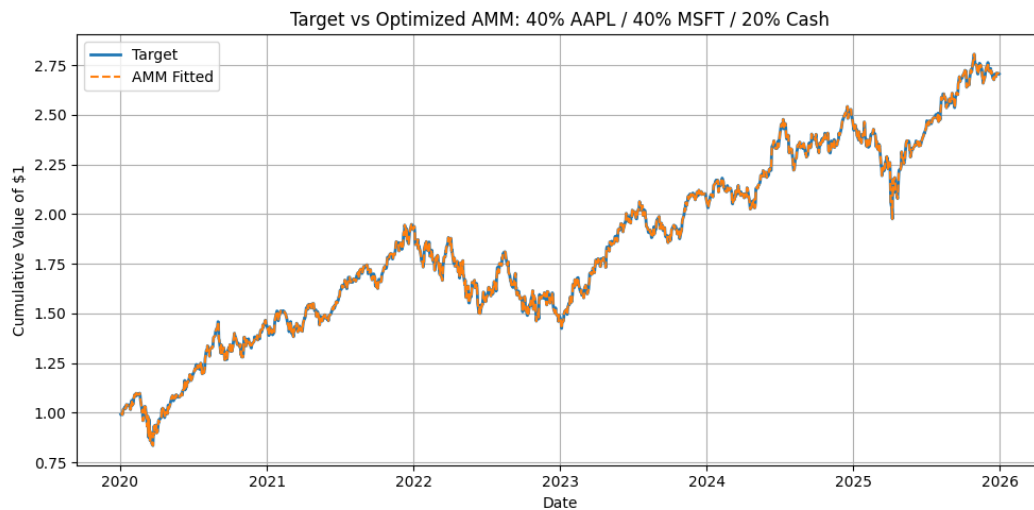


Figure 3: Target vs. optimized AMM: 40% AAPL / 40% MSFT / 20% cash.

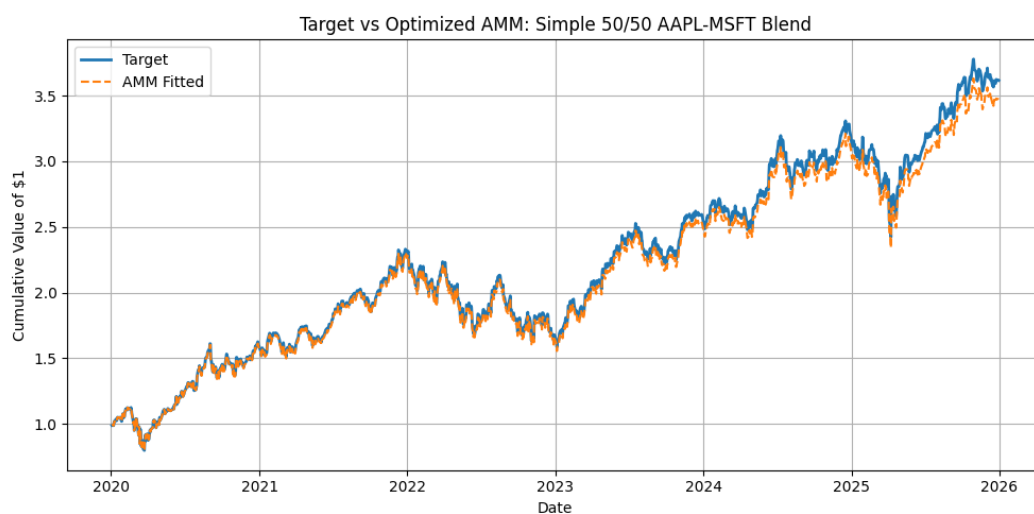


Figure 4: Target vs. optimized AMM: simple 50/50 AAPL-MSFT arithmetic blend.

Figure ?? shows the Newton solver convergence.

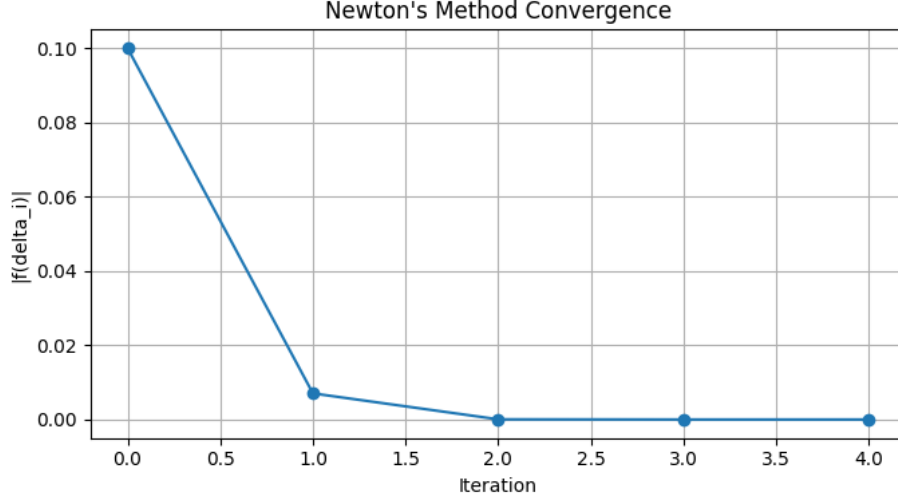


Figure 5: Newton’s method convergence: $|f(\delta_i)|$ per iteration (log scale).

10 Validation

We validate the results in four ways:

- **Weight recovery.** For the `sqrt_aapl` and `mixed_40_40` targets, the optimizer recovers the exact generating weights. This confirms the implementation is correct.
- **Tracking error.** Max absolute log-return error is 0 for geometric targets and 1.1×10^{-3} for the arithmetic target, which is expected.
- **Constraint satisfaction.** All solved weights are non-negative and sum to 1.
- **Newton convergence.** The solver reaches $|f(\delta_i)| < 10^{-10}$ in 4 iterations for the example case. The convergence plot shows the residual dropping rapidly, consistent with the quadratic convergence rate expected from Newton’s method near a smooth zero.

All results use real AAPL and MSFT historical return data, which makes the test more realistic even if the AMM model is simplified.

11 Key Insights

- AMM pool weights directly control log-return exposure. A weight of w_A on AAPL captures exactly w_A of AAPL’s log-return each day.
- A 50/50 AAPL/cash pool naturally produces square-root return behavior. This is a property of the geometric mean formula, not a design choice.

- Long-only constraints produce fractional exposure, not leverage. The pool always holds some cash, which absorbs the unused weight.
- Geometric targets (defined in log-return space) can be matched exactly by the AMM. Arithmetic targets (defined in dollar terms) can only be approximated, because the log of an arithmetic average is not a linear combination of logs.
- Newton’s method is effective for the price-alignment problem in the simplified two-asset case. The function $f(\delta_i)$ is smooth and strictly monotone, so Newton converges quickly from a zero initial guess.

12 Limitations

- The Newton solver assumes a zero-fee AMM. Real AMMs charge swap fees, which shift the equilibrium trade size.
- The model ignores transaction costs, bid-ask spreads, and slippage. In practice these matter for the profitability of arbitrage.
- We use historical AAPL and MSFT stock returns as stand-ins for AMM pool token returns. A real AMM pool holds tokens whose prices are set by the pool itself, not by an external exchange.
- Long-only weight constraints mean the model cannot represent leverage or short positions.
- The external market price is taken as given. In a real setting, the external price is also stochastic, and the problem would need to account for timing and uncertainty.
- Results are sensitive to the chosen date range. Different periods may produce different optimal weights.

13 Conclusion

This project asked two questions: can AMM weights create controlled fractional return exposure, and can Newton’s method solve the price-alignment trade?

The answer to the first question is yes, at least for targets defined in log-return space. The constrained least-squares optimizer recovered the generating weights exactly for geometric targets and closely approximated an arithmetic blend. The optimization model is convex, the constraints are linear, and the problem is well-posed.

The answer to the second question is also yes, in the simplified zero-fee two-asset setting. Newton’s method found the equilibrium trade in four iterations with residual below 10^{-10} .

The project connects three ideas — AMM pool design, constrained optimization, and numerical root-finding — in a focused experiment. The results are consistent with theory and the implementation is straightforward. The main limitation is that both the optimization and the Newton solver are built on a simplified AMM model that omits real-world frictions.

14 Future Work

- Add swap fees to the Newton solver and study how fees affect the equilibrium trade size.
- Extend the Newton solver to multi-asset pools with more than two tokens.
- Use real AMM pool reserve data instead of stock return proxies.
- Study dynamic or time-varying weights rather than fixed allocations.
- Simulate jump-diffusion return processes to test the optimizer under heavier-tailed returns.
- Compare SLSQP to a closed-form least-squares solution (since the unconstrained version of the problem has an analytic answer) and to projected gradient descent.
- Study how the fractional exposure property behaves during market crashes or high-volatility periods.